

Android push cannot send — FCM provider fails to initialise

Teamder · 23 August 2026 · workspace `33b93794-3468-4241-aeaf-d752166ae066`

The problem

A push campaign to a single recipient fails at the provider. The recipient row:

```
{
  "email": "matanlevi95@gmail.com",
  "status": "failed",
  "failureReason": "FCM requires firebase-admin to be installed. Run: npm install fire
}
```

The campaign itself reports nothing wrong — `status: active`, `failureReason: null`, `stats: {"queued": 1}`, `sendVolumeSent: 0`. The failure is visible only on the recipient row.

Our side is fully configured, and verified independently

- Firebase service account uploaded; the app page shows **Configured** (project soccer-app-52b6b). Sender ID `559368532219` saved.
 - The service account is known-good: we sent **3 of 3** notifications with it straight to `fcm.googleapis.com/v1/projects/soccer-app-52b6b/messages:send` and all three were accepted. The credential works.
 - It is scoped to `roles/firebasecloudmessaging.admin` — 9 permissions, all messaging. Not a full Firebase admin key.
 - Audience resolves to exactly 1 user out of 607 (verified by scanning every user for the targeting attribute).
-

The error message is probably not the real cause

`firebase-admin` is declared in the backend's `package.json` as `^14.0.0`. And `fcm-provider.ts` wraps the whole initialisation in one `try`:

```
const admin = require('firebase-admin');
this.adminApp = admin.initializeApp({
  credential: admin.credential.cert(config.serviceAccount),
  projectId: config.projectId,
}, `fcm-${Date.now()}-...`);
} catch (error) {
  throw new Error('FCM requires firebase-admin to be installed. Run: npm install firebase-admin');
}
```

So a failure in `admin.credential.cert(config.serviceAccount)` — a malformed or wrongly-shaped stored service account — reports itself as a missing package. Two candidates:

1. `firebase-admin` is genuinely absent from the deployed image (declared but not installed / pruned in the build).
2. The uploaded JSON is stored in a shape `cert()` rejects (e.g. kept as a string rather than a parsed object).

The dashboard reads `project_id` back out of the stored credential, which leans towards (1). Either way, please log `error` in that catch — the current message can only ever describe one of the causes it reports.

Two other findings from the same trace

Bulk-imported push tokens are permanently undeliverable, silently. `push-notification.service.ts` filters devices with `query['deviceInfo.appId'] = appId`. Device rows created through the REST API with a `jry_live_` key carry no `deviceInfo.appId` — that field is stamped only on `POST /v1/push/register` with an app-scoped `jry_sdk_` key. We migrated ~400 tokens through the REST path; every one of them was invisible to the push worker, and the only symptom was `"no active push devices"` on the recipient row. Re-registering through the SDK key fixed it. Anyone migrating tokens in bulk will hit this, and nothing warns them.

Re-registering an existing token returns an internal error. `POST /v1/push/register` with a token already present on a device row returns `{"success": false, "error": "An internal error occurred"}`; a brand-new token succeeds. Registration is idempotent by design and runs on every app launch, so this fails for every returning user. Unregistering the token first and then re-registering works — and creates a duplicate row rather than updating the existing one.

Not blocking us, but worth noting

`POST /campaigns` accepts `appId` , and a push campaign needs it (or `channelConfig.platforms`) to reach a consumed queue. But `GET /apps` returns **401** for a `jry_live_` key and no SDK endpoint exposes the app id, so the value is unobtainable outside a dashboard session. We only learned ours — `582c204f-327e-4d76-9703-b8655454b887` — by reading it back out of a device row after re-registering a token. Exposing app id + platform to the API key would close that loop.